TOURVAA PLATFORM: SYSTEM ARCHITECTURE & DIAGRAMS REFERENCE

03 — System Architecture & Component Interaction

1. Architectural Philosophy

Tourvaa is built upon clean architecture principles:

1. **Decoupled Client & Server**: The Next.js frontend acts purely as a presentation and edge-routing layer, communicating with the FastAPI backend via JSON REST APIs and WebSocket streams.

2. **Layered Backend Separation**: Routers handle HTTP transport and validation; domain services encapsulate business logic; SQLAlchemy models define persistence; background workers manage asynchronous tasks.

3. **Escrow Booking Workflow**: Payment success does not equal final booking confirmation. Bookings follow an explicit supplier acceptance and Tourvaa operational approval lifecycle.

---

2. High-Level Architecture Diagram

```
flowchart TB
    USER[User / Browser] ⟶|HTTPS| EDGE[Next.js 16 Edge / Proxy]
    USER ⟶|WSS| BACKEND[FastAPI Backend :8000]

    EDGE ⟶|Proxies /api & /storage| BACKEND

    BACKEND ⟶ AUTH[Dual-Domain Auth: Admin & Normal]
    AUTH ⟶ SERVICES[56 Business Domain Services]
    SERVICES ⟶ DB[(MySQL 8.0+ Database)]
    SERVICES ⟶ REDIS[(Redis Cache & Distributed Locks)]
    SERVICES ⟶ FS[(Private Disk Storage)]

    SERVICES ⟶ GATEWAYS[Stripe / PayPal]
    SERVICES ⟶ VIATOR[Viator Partner API]
    SERVICES ⟶ SMTP[SMTP Email Relay]
    SERVICES ⟶ PUSH[WebPush Push Server]
```

Detailed architectural diagrams and component walkthroughs are available in:

- System Architecture Diagram
- Frontend Component Architecture
- Backend Architecture & Request Lifecycle
- Module Dependencies Diagram

---

05 — Frontend Architecture & Component Design

1. Next.js 16 App Router Structure

The frontend is built on **Next.js 16.4.3** (locked in `package.json`; upgraded from initial 15.5 baseline) utilizing the React 19.2.4 App Router paradigm. It is divided into logical route groups:

1. `%28public%29/`: Publicly accessible tour discovery, destination search, checkout, blog articles, and institutional pages.
2. `admin/`: Back-office administration for platform operators, tour review workflows, and financial settings.
3. `customer/`: Consumer portal for viewing booked itineraries, passenger rosters, and payments.
4. `supplier/`: Tour supplier management console for managing departures, pricing, and bookings.
5. `agent/`: B2B wholesale agent console for retail client reservations and commissions.
6. `affiliate/`: Affiliate partner dashboard for referral short-links, clicks, and wallet payouts.

---

2. Server-Side Route Guard (src/proxy.ts)

Next.js 16 replaced the traditional `middleware.ts` convention with `proxy.ts`. Tourvaa's proxy executes before any protected page HTML is sent to the browser:

- Inspects incoming requests for the `tourvaa_access` session cookie on all protected path prefixes (`/admin`, `/customer`, `/agent`, `/supplier`, `/affiliate`).
- Unauthenticated requests are immediately redirected to `/login` or `/admin/login` with the original URL preserved in `?redirect=`.
- Normalizes trailing slashes with permanent 308 redirects to avoid duplicate routing.

---

3. Global Context Providers

1. **`AuthProvider` (`src/providers/AuthProvider.tsx`)**:
   - Manages active user authentication state, current role claims, and user profile data.
   - Listens for token invalidation events (e.g. password resets or remote session termination).
2. **`PublicSettingsProvider` (`src/providers/PublicSettingsProvider.tsx`)**:
   - Fetches and caches global platform configuration, currency exchange rates, company branding, and active payment gateway toggles.
3. **`TravelStoreProvider` (`src/providers/TravelStoreProvider.tsx`)**:

- Manages client-side shopping cart state, tour comparison lists, and customer wishlist items backed by `localStorage`.

---

4. API Communication & Proxy Rewrites

All browser HTTP requests to `/api/*` and `/storage/*` are proxied to the backend via `next.config.ts` rewrites:

```
{
  source: "/api/:path*",
  destination: `${apiProxyTarget}/api/:path*`,
}
```

This ensures that the frontend communicates with the backend as a **same-origin service**, allowing secure, `httpOnly`, `SameSite=Lax` cookies to be exchanged without exposing raw JWTs to client-side JavaScript.

The complete frontend route inventory is cataloged in `inventories/frontend-routes.md`.

---

06 — Backend Architecture & Service Layer Design

1. Application Architecture

The backend is built with **FastAPI 0.139.2** (locked in `requirements.txt`; upgraded from initial 0.115 baseline) running on **Python 3.12**. It uses an asynchronous service layer backed by SQLAlchemy 2.0 ORM and PyMySQL.

---

2. Startup Lifecycle & Production Verification (_lifespan)

FastAPI's async lifespan context manager in `app/main.py` enforces strict startup safety gates:

1. `validate_database_migration_head()`: Compares running database heads against Alembic code heads. Aborts if migrations are pending.

2. `validate_production_config()`: In production (`APP_ENV ≠ "development"`), ensures debug is disabled, JWT secrets are >= 32 characters, and webhook secrets are present.

3. `validate_worker_concurrency()`: Enforces that `REDIS_URL` is set whenever `WEB_CONCURRENCY > 1`.

4. `validate_pdf_rendering_dependencies()`: Verifies that ReportLab and Pillow are functional before accepting traffic.

5. `run_seed()`: Seeds default roles, permissions, and email templates if missing.

6. `start_background_jobs()`: Launches asynchronous worker coroutines with distributed locks.

---

3. Middleware Stack

Requests pass through four specialized middlewares:

1. **CORS Middleware**: Validates allowed origins against `ALLOWED_ORIGINS` (wildcards prohibited in production).

2. **ApiNoStore Middleware**: Emits `Cache-Control: no-store` to prevent caching of sensitive customer or financial data.

3. **CsrfMiddleware**: Defends against cross-site request forgery by validating `Origin` and `Referer` headers on state-mutating HTTP methods.

4. **Error Handlers**: Traps validation errors (`RequestValidationError`) and internal exceptions, formatting them into standardized JSON error responses.

---

4. Background Job Coroutines

12 background tasks run concurrently within the FastAPI event loop, acquiring distributed locks via Redis (`app/utils/distributed_lock.py`):

- `_expire_stale_bookings_loop`: Sweeps every 15 minutes; releases unpaid seats held longer than 60 minutes.
- `_payment_hold_loop`: Sweeps every 60 minutes; releases pre-authorization card holds exceeding the 4-day supplier decision window.
- `_payment_capture_retry_loop`: Sweeps every 15 minutes; retries failed captures on confirmed bookings.
- `_email_retry_loop`: Sweeps every 15 minutes; retries failed lifecycle emails using exponential backoff (1m, 5m, 25m).
- `_push_retry_loop`: Sweeps every 15 minutes; retries failed WebPush deliveries.
- `_balance_due_reminder_loop`: Sweeps every 60 minutes; dispatches payment reminders for upcoming booking balances.
- `_tour_availability_reminder_loop`: Sweeps every 60 minutes; alerts suppliers when tour departure schedules are nearing expiration.
- `_wishlist_reminder_loop`: Sweeps every 24 hours; notifies customers of seat availability on wishlisted tours.
- `_chat_retention_loop`: Sweeps every 24 hours; purges chat logs older than 90 days.
- `_data_retention_loop`: Sweeps every 24 hours; prunes stale user sessions and email delivery logs.

---

08 — Database Schema & Data Architecture

1. Database Engine & ORM Configuration

- **Database Engine**: MySQL 8.0+ or MariaDB using the **InnoDB storage engine** for full ACID compliance and row-level locking.
- **ORM Framework**: SQLAlchemy 2.0 utilizing Declarative Base (`Base = declarative_base()`).
- **Connection Pooling**: Managed via `QueuePool` with configurable pool size, max overflow, and automated pool recycling to prevent stale MySQL disconnects.
- **Character Set**: `utf8mb4` with `utf8mb4_unicode_ci` collation supporting international multi-lingual text and emoji characters.

---

2. Relational Schema Statistics

- **Total Tables**: **138 database tables**
- **Migration Revisions**: **170 Alembic migration files** (dating from `2026-06-15` to `2026-10-28`).
- **Foreign Key Density**: High relational integrity with cascading deletion restricted on financial and audit records.

---

3. Core Relational Domains

1. **Identity & Access**: `users` , `roles` , `permissions` , `role_permissions` , `user_roles` , `user_sessions` , `login_history` , `audit_logs` .

2. **Geographical Master**: `countries` , `states` , `cities` , `cms_country_pages` , `cms_favourite_countries` .

3. **Tour Catalog**: `tours` , `tour_overviews` , `tour_itineraries` , `tour_pricing` , `tour_calendar` , `tour_unavailable_dates` , `tour_inclusions` , `tour_exclusions` , `tour_highlights` , `tour_gallery_images` , `tour_categories` , `tour_subcategories` .

4. **Tour Versioning**: `tour_versions` , `tour_review_comments` .

5. **Booking Hub**: `bookings` , `booking_travellers` , `booking_optional_activities` , `booking_accommodations` , `booking_extensions` , `booking_status_history` , `booking_communications` , `booking_calendar_events` .

6. **Financial & Checkout**: `payments` , `payment_transactions` , `payment_holds` , `checkout_sessions` , `invoices` , `invoice_items` .

7. **Partner Ledgers**: `suppliers` , `supplier_ledgers` , `supplier_payouts` , `agents` , `agent_ledger` .

8. **Affiliate Engine**: `affiliates` , `affiliate_links` , `affiliate_clicks` , `affiliate_attributions` , `affiliate_commission_rules` , `affiliate_conversions` , `affiliate_payouts` , `affiliate_wallet_transactions` .

9. **CMS & Engagement**: `cms_homepage_banners`, `cms_popular_destinations`, `cms_popular_tours`, `cms_blogs`, `cms_customer_reviews`, `cms_policies`.

10. **Communications & AI**: `conversations`, `messages`, `notifications`, `notification_deliveries`, `chat_faqs`, `chat_sessions`, `chat_messages`, `chat_embeddings`.

The complete schema reference of all 138 tables and columns is cataloged in `inventories/database-tables.md`.

System Architecture Diagram & Walkthrough

This document illustrates the end-to-end system architecture of the Tourvaa travel platform, showing the multi-tier boundaries between users, frontend edge layers, backend API services, asynchronous workers, persistence layers, and external service providers.

1. High-Level Architecture Diagram

```
flowchart TB
    subgraph CLIENTS["Client Access Tier"]
        BROWSER["Web Browser / Mobile Client<br/>(Desktop, Tablet, Mobile)"]
    end

    subgraph EDGE["Edge & Routing Tier (Next.js 16)"]
        PROXY["Next.js Proxy (../../src/proxy.ts)<br/>Cookie Verification & Route Shield"]
        REWRITE["Next.js Route Proxy (next.config.ts)<br/>Rewrites /api and /storage"]
        SSR["Server Component Rendering<br/>(React 19 / Next.js)"]
        SPA["Client-Side Portals (SPA)<br/>Admin / Supplier / Agent / Affiliate / Customer"]
    end

    subgraph API_TIER["FastAPI Application Tier (Port 8000)"]
        UVICORN["Uvicorn ASGI Server<br/>(Multi-Worker Concurrency)"]
        MIDDLEWARE["Middleware Stack<br/>(CORS, ApiNoStore, CSRF, ErrorHandlers)"]
        ROUTERS["API Routers (51 Modular Routers)<br/>(/api/* and /r/*)"]
        AUTH_GUARD["Authentication & RBAC<br/>(Dual JWT Domain & RolePermission Checker)"]
        SERVICES["Domain Services Layer (56 Services)<br/>(Bookings, Payments, Tours, Ledgers, etc.)"
        WS_HANDLER["WebSocket Hub (/api/ws/messages)<br/>(Realtime Messaging & Live Notifications)"]
    end

    subgraph BACKGROUND["Background Workers & Scheduled Sweeps"]
        STALE_SWEEP["Stale Booking Expiry (15m)<br/>Releases Unpaid Calendar Seats"]
        HOLD_SWEEP["Payment Hold Release (60m)<br/>Releases Expired Gateway Authorizations"]
        CAPTURE_SWEEP["Payment Capture Retry (15m)<br/>Retries Accepted Booking Captures"]
        EMAIL_RETRY["Lifecycle Email Retry (15m)<br/>Exponential Backoff 1m/5m/25m"]
        REMINDERS["Booking & Balance Reminders (60m)<br/>Automated Traveller & Agent Alerts"]
        RETENTION["Data Retention Pruning (24h)<br/>Prunes Old Sessions & Chat Logs"]
    end

    subgraph STORAGE["Persistence & Data Tier"]
        MYSQL[("MySQL 8.0+ Database<br/>(InnoDB Engine, 138 Tables)")]
        REDIS[("Redis Cache & Pub/Sub<br/>(Distributed Lock & WS Relay)")]
        LOCAL_DISK[("Private Disk Storage<br/>(private-docs: Invoices, Itineraries, Docs)")]
    end

    subgraph EXTERNAL["External Third-Party Services"]
        STRIPE["Stripe Payment Gateway<br/>(Manual Capture & Webhooks)"]
        PAYPAL["PayPal REST API<br/>(Authorize & Capture Orders)"]
        CLOUDINARY["Cloudinary Media CDN<br/>(Public Tour & CMS Images)"]
        VIATOR["Viator Partner API<br/>(External Day Trips & Experiences)"]
        SMTP["SMTP Mail Server<br/>(Lifecycle & Transactional Emails)"]
        WEBPUSH["WebPush Push Services<br/>(FCM / Mozilla VAPID)"]
        GEOIP["GeoIP Provider (ip-api / ipapi)<br/>(Currency & Locale Auto-Detection)"]
    end

    %% Client flows
    BROWSER ──>|HTTPS Requests| PROXY
    BROWSER ──>|Direct WebSocket Upgrade (wss://)| UVICORN

    %% Edge flows
    PROXY ──> SSR
    PROXY ──> SPA
    SSR ──> REWRITE
    SPA ──> REWRITE
    REWRITE ──>|HTTP Forwarding| UVICORN
```

```
    %% API Tier flows
    UVICORN ⟶ MIDDLEWARE
    MIDDLEWARE ⟶ ROUTERS
    ROUTERS ⟶ AUTH_GUARD
    AUTH_GUARD ⟶ SERVICES
    ROUTERS ⟶ WS_HANDLER

    %% Persistence flows
    SERVICES ⟶|SQLAlchemy 2.0 Pool| MYSQL
    SERVICES ⟶|Distributed Locks & Keys| REDIS
    SERVICES ⟶|Direct FS Read/Write| LOCAL_DISK
    WS_HANDLER ⟶|Pub/Sub Channel ws:messages| REDIS

    %% Background worker flows
    BACKGROUND ⟶|acquire_sweep_lock| REDIS
    BACKGROUND ⟶|Scheduled Task Execution| SERVICES

    %% External integration flows
    SERVICES ⟶|REST API & Webhooks| STRIPE
    SERVICES ⟶|REST API & Webhooks| PAYPAL
    SERVICES ⟶|Asset Uploads| CLOUDINARY
    SERVICES ⟶|Partner API Requests| VIATOR
    SERVICES ⟶|SMTP TLS Connection| SMTP
    SERVICES ⟶|VAPID HTTP/2 Push| WEBPUSH
    SERVICES ⟶|REST Query| GEOIP
```

---

2. Component Descriptions & Boundaries

2.1 Client Access Tier

Users connect via standard modern web browsers on desktop, tablet, and mobile devices. All public web traffic terminates at the Next.js edge router over HTTPS. WebSocket connections upgrade directly to the backend origin (`NEXT_PUBLIC_WS_URL`) over secure WebSockets (`wss://`).

2.2 Edge & Routing Tier (Next.js 16)

- **`src/proxy.ts`** : Implements Next.js 16's server-side proxy convention (formerly middleware). Checks for the presence of the `tourvaa_access` session cookie on all protected portal routes (`/admin/*` , `/customer/*` , `/supplier/*` , `/agent/*` , `/affiliate/*`) before serving React Server Components, preventing HTML flashes of unauthorized UI.
- **`next.config.ts`** : Provides path rewriting that transparently proxies client browser calls to `/api/:path*` and `/storage/:path*` directly to `API_PROXY_TARGET` . It also enforces HTTP Content Security Policy (CSP), frame guards, and client max body size limits (`60mb` for video uploads).

2.3 FastAPI Application Tier

- **Uvicorn Server**: Production ASGI server configured with single or multiple workers. Supports process-level concurrency and HTTP/1.1 and WebSocket protocols.
- **Middleware Pipeline**:
- `setup_cors` : Strictly enforces permitted origins from `ALLOWED_ORIGINS` with credentials enabled.

- `ApiNoStoreMiddleware`: Appends `Cache-Control: no-store` headers to eliminate stale caching of dynamic user data.
- `CsrfMiddleware`: Defends state-modifying requests (`POST`, `PUT`, `DELETE`, `PATCH`).
- `register_error_handlers`: Normalizes uncaught exceptions and HTTP validation errors into unified JSON error structures.
- **Dual-Domain Authentication**: Authenticates JWTs using segregated signing keys for administration staff (`admin` domain) vs external portal users (`normal` domain).
- **Service Layer**: Houses 56 dedicated business domain services that decouple database queries and external APIs from the router handlers.

2.4 Background Scheduler

FastAPI lifespan background task loops run concurrently without external worker dependencies (like Celery), leveraging `app/utils/distributed_lock.py` backed by Redis:

- Stale unpaid booking seat release (15-minute sweep)
- Payment hold release after 4-day decision window (60-minute sweep)
- Payment capture retry following supplier acceptance (15-minute sweep)
- Durable email delivery retry with exponential backoff (15-minute sweep)
- Customer balance-due reminders and travel date notifications (60-minute sweep)
- Data retention pruning for sessions and chat messages (24-hour sweep)

2.5 Data & Storage Tier

- **MySQL 8.0+ Database**: Stores the relational schema across 138 tables with InnoDB ACID guarantees.
- **Redis Cache**: Backs distributed locking (`acquire_sweep_lock`), rate-limiting buckets, WebSocket single-use ticket store, and cross-worker pub/sub messaging.
- **Private Disk Storage**: Dedicated filesystem root (`private-docs/`) segregated from public static storage, holding confidential documents (supplier business licenses, agent agreements, generated invoices, and booking receipts).

2.6 External Integrations

Provides modular connectivity to Stripe (holds & captures), PayPal (orders & authorizations), Cloudinary (CDN media), Viator (third-party day tours), SMTP (transactional mail), WebPush (browser push notifications), and GeoIP (browsing currency localization).

Frontend Component Architecture & Navigation Flow

This document details the Next.js 16 App Router component architecture, portal layouts, route guards, state providers, and client-server boundaries of the Tourvaa frontend (`tourvaa-admin-frontend`).

1. Frontend Architecture Diagram

```
graph TD
    subgraph APP_ROOT["Next.js App Root (../../src/app/)"]
        ROOT_LAYOUT["RootLayout (../../src/app/layout.tsx)<br/>Fonts: Onest, Inter | HTML Shell"]
        PROXY["Server Proxy Guard (../../src/proxy.ts)<br/>Checks tourvaa_access Session Cookie"]

        subgraph PROVIDERS["Global Context Providers (../../src/providers/)"]
            AUTH_PROV["AuthProvider<br/>(User State, Login/Logout, Token Version)"]
            SETTINGS_PROV["PublicSettingsProvider<br/>(Currency Master, Branding, Site Settings)"]
            STORE_PROV["TravelStoreProvider<br/>(Cart, Wishlist, Compare, Draft Bookings)"]
        end
    end

    subgraph PORTAL_SHELLS["Portal Layout Shells & Route Guards"]
        PUBLIC_SHELL["%28public%29 Layout<br/>(Header, Nav, Footer, Elfsight Widget, Chatbot)"]
        ADMIN_SHELL["admin/ Layout & AdminRouteGuard<br/>(Sidebar, TopBar, RBAC Permission Gate)"]
        CUSTOMER_SHELL["customer/ Layout & ProtectedRoute<br/>(Customer Portal Navigation, Account Ta
        SUPPLIER_SHELL["supplier/ Layout & ProtectedRoute<br/>(Supplier Nav, Approval Status Banner)"
        AGENT_SHELL["agent/ Layout & ProtectedRoute<br/>(Agent B2B Nav, Balance Alerts)"]
        AFFILIATE_SHELL["affiliate/ Layout & ProtectedRoute<br/>(Affiliate Nav, Wallet Balance Bar)"]
    end

    subgraph PAGE_COMPONENTS["Feature Page Modules (../../src/components/)"]
        TOURS_UI["Tours & Catalog UI<br/>(TourGrid, TourDetails, TourWizard, TourCalendar)"]
        BOOKING_UI["Booking & Checkout UI<br/>(CheckoutForm, TravellerList, PaymentModal)"]
        CMS_UI["CMS & Content Management<br/>(BannerManager, BlogEditor, PolicyManager)"]
        FINANCE_UI["Invoices, Ledgers & Payouts<br/>(InvoiceList, LedgerTable, PayoutRequestModal)"]
        MESSAGING_UI["Messaging & Support UI<br/>(ThreadView, SocketPresence, ChatbotWidget)"]
        REPORTS_UI["Analytics & Reporting UI<br/>(Recharts Visualizations, DateFilters, ExcelExport)"
    end

    subgraph API_CLIENT["API Client Layer (../../src/lib/api/)"]
        CLIENT_AXIOS["Axios Instance (../../src/lib/api/client.ts)<br/>withCredentials: true, Automat
        PUBLIC_AXIOS["Public Axios Instance (../../src/lib/api/publicClient.ts)<br/>Cached GET Reques
        WS_CLIENT["WebSocket Hook (../../src/hooks/useMessagingSocket.ts)<br/>Ticket Exchange & Auto-
    end

    %% Hierarchy flows
    ROOT_LAYOUT --> PROXY
    PROXY --> PROVIDERS
    PROVIDERS --> PORTAL_SHELLS

    PUBLIC_SHELL --> TOURS_UI
    PUBLIC_SHELL --> BOOKING_UI
    PUBLIC_SHELL --> MESSAGING_UI

    ADMIN_SHELL --> CMS_UI
    ADMIN_SHELL --> FINANCE_UI
    ADMIN_SHELL --> REPORTS_UI
    ADMIN_SHELL --> TOURS_UI

    CUSTOMER_SHELL --> BOOKING_UI
    CUSTOMER_SHELL --> FINANCE_UI
    CUSTOMER_SHELL --> MESSAGING_UI

    SUPPLIER_SHELL --> TOURS_UI
    SUPPLIER_SHELL --> FINANCE_UI
    SUPPLIER_SHELL --> BOOKING_UI
```

```
    AGENT_SHELL ⟶ BOOKING_UI
    AGENT_SHELL ⟶ FINANCE_UI

    AFFILIATE_SHELL ⟶ FINANCE_UI

    %% Client data bindings
    PAGE_COMPONENTS ⟶ CLIENT_AXIOS
    PAGE_COMPONENTS ⟶ PUBLIC_AXIOS
    PAGE_COMPONENTS ⟶ WS_CLIENT
```

---

2. Component Hierarchy & Navigation Flow

2.1 Route Guarding Strategy

The application employs **two-tier defense-in-depth route guarding**:

1. **Server-Side Edge Guard (`src/proxy.ts`)**:

- Executes before React Server Components render.
- Evaluates incoming paths against `PROTECTED_PREFIXES` (`/admin` , `/customer` , `/agent` , `/supplier` , `/affiliate`).
- If the request lacks the `tourvaa_access` session cookie, it immediately redirects to `/login` (or `/admin/login`) with `?redirect=<target_path>` .
- Also normalizes trailing slashes to prevent duplicate routing loops.

2. **Client-Side Component Guards (`ProtectedRoute` & `AdminRouteGuard`)**:

- Inspects the decoded user claims in `AuthProvider` .
- Checks role assignment (`super-admin` , `admin` , `supplier` , `agent-reseller` , `customer` , `affiliate`).
- For administrative pages, cross-references granular permissions (e.g. `tours.view` , `bookings.edit`) via `useAdminModules` before mounting sensitive panels.
- For suppliers and agents, inspects `approval_status` and blocks operational views if approval is still pending.

2.2 Global State Management

- **`AuthProvider`** : Manages user authentication state, active portal mapping, and account session tokens. Intercepts backend logout and password-change token invalidations.
- **`PublicSettingsProvider`** : Synchronizes platform-level configuration (company contact information, active currency lists, exchange rates, and branding metadata) across all public pages.
- **`TravelStoreProvider`** : Manages browser-local shopping cart items, tour comparison selections, saved wishlist items, and guest checkout draft payloads using `localStorage` .

2.3 Realtime Messaging Hook (useMessagingSocket)

- Requests single-use authentication tickets via `POST /api/messaging/ws/ticket` through the same-origin Next.js proxy.
- Establishes a direct WebSocket connection to `NEXT_PUBLIC_WS_URL/api/ws/messages?ticket=<token>`.
- Maintains heartbeat pings every 30 seconds.
- Dispatches realtime UI updates for new booking messages, admin notices, and unread badge counters.

Backend Architecture & Request-Response Lifecycle

This document describes the internal architecture of the Tourvaa FastAPI backend (`tourvaa-admin-backend`), detailing application startup, middleware sequencing, route dispatching, dependency injection, and data persistence.

1. Backend Architecture & Request Flow Diagram

```
sequenceDiagram
    autonumber
    actor Client as Browser / Client
    participant Proxy as Next.js / Reverse Proxy
    participant Uvicorn as Uvicorn ASGI Server
    participant Middleware as Middleware Stack
    participant Router as API Router (/api/*)
    participant Auth as Auth & RBAC Dependency
    participant Service as Business Service Layer
    participant DB as SQLAlchemy / MySQL DB
    participant Background as Background Tasks

    Note over Uvicorn, DB: Application Startup (Lifespan Context)
    Uvicorn->>DB: validate_database_migration_head()
    Uvicorn->>Uvicorn: validate_production_config() & validate_worker_concurrency()
    Uvicorn->>DB: run_seed() (Roles, Permissions, Email Templates)
    Uvicorn->>Background: start_background_jobs() (Asyncio Task Loops)

    Note over Client, DB: Inbound Request Lifecycle
    Client->>Proxy: HTTP Request (Headers, Cookies, Payload)
    Proxy->>Uvicorn: Forward Request
    Uvicorn->>Middleware: Execute setup_cors()
    Middleware->>Middleware: Execute ApiNoStoreMiddleware (Cache-Control: no-store)
    Middleware->>Middleware: Execute CsrfMiddleware (Origin / Referer Validation)
    Middleware->>Router: Route Match (/api/v1/...)

    alt Route requires Authentication
        Router->>Auth: Depends(get_current_user)
        Auth->>Auth: Extract 'tourvaa_access' cookie or 'Bearer' token
        Auth->>Auth: decode_auth_token() (Validate admin or normal domain key)
        Auth->>DB: Check User token_version & UserSession JTI
        Auth->>Auth: Verify user.is_active and approval_status
        Auth-->>Router: Inject User object
    end

    alt Route requires Specific Permission
        Router->>Auth: Depends(require_permission("bookings.edit"))
        Auth->>DB: Query RolePermission for User Role
        Auth-->>Router: Access Granted or 403 Forbidden
    end

    Router->>Service: Call Domain Service (e.g. bookings.update_booking_status)
    Service->>DB: Begin DB Transaction (SessionLocal)
    Service->>DB: Query / Mutate Models (SQLAlchemy ORM)
    Service->>DB: Commit DB Transaction

    opt Asynchronous Event Triggered
        Service->>Background: Schedule Post-Commit Task (e.g. Email / Push / WS Event)
    end

    Service-->>Router: Return Pydantic Response Schema
    Router-->>Middleware: 200 OK / Response JSON
    Middleware-->>Proxy: Set Response Headers
    Proxy-->>Client: Deliver HTTP Response

    Note over Background, DB: Post-Response Asynchronous Execution
```

```
    Background-->>DB: NotificationDelivery / SMTP Dispatch
    Background-->>Client: WebPush VAPID / WebSocket Broadcast
```

---

2. Request-Response Lifecycle Stages

Stage 1: Lifespan Startup & Self-Healing Guards

When the FastAPI process boots, `_lifespan` in `app/main.py` executes:

1. **Migration Verification**: Compares Alembic code heads to running database heads. Discrepancies abort the server to prevent queries against outdated schemas.

2. **Production Hardening Checks**: Validates strong secrets, verifies debug mode is off, and ensures separate admin and normal domain keys.

3. **Concurrency Guard**: Ensures `REDIS_URL` is set if `WEB_CONCURRENCY > 1`.

4. **Automated Seeding**: Executes `run_seed()` to backfill any newly introduced roles, admin modules, or email templates.

5. **Background Sweeps Launch**: Starts 12 distinct background coroutines for payment hold releases, booking expirations, balance alerts, and email retries.

Stage 2: Middleware Stack

Every request passes through three custom middlewares:

1. **CORS Middleware**: Evaluates origin against `ALLOWED_ORIGINS` (fails closed if wildcard in production).

2. **ApiNoStore Middleware**: Enforces `Cache-Control: no-store, no-cache, must-revalidate, max-age=0` to ensure user sessions are never cached by intermediate proxies or browser history.

3. **CSRF Middleware**: Validates that state-modifying requests (`POST`, `PUT`, `DELETE`, `PATCH`) carry an `Origin` or `Referer` matching allowed platform hosts.

Stage 3: Routing & Dependency Injection

- API routers are segregated into 4 logical groups (`CORE_ROUTERS`, `PARTNER_AND_CUSTOMER_ROUTERS`, `CONTENT_AND_TOUR_ROUTERS`, `OPERATIONS_ROUTERS`) and registered under `/api`.
- Request schemas are validated using **Pydantic v2**, rejecting malformed data with `422 Unprocessable Entity` before hitting business logic.
- Authentication dependencies (`get_current_user`, `require_permission`) inspect the JWT domain, session table, and role permission mappings.

Stage 4: Business Service Layer & Transaction Boundaries

- Router endpoints never perform raw database queries directly. They delegate to dedicated service modules in `app/services/`.
- All database modifications take place within explicit SQLAlchemy session boundaries.

- Critical operations (such as payment captures and supplier acceptance) utilize transaction commits before triggering post-commit notifications.

Stage 5: Asynchronous Post-Commit Execution

- Emails, push notifications, and WebSocket broadcasts are never performed synchronously inside request threads.
- Instead, services enqueue records into `notification_deliveries` and trigger non-blocking `BackgroundTasks` to ensure that external network delays never slow down user requests.

Database Entity-Relationship Diagram (ERD) & Schema Overview

This document presents the Entity-Relationship Diagram (ERD) of the core database entities in the Tourvaa platform, modeling the relationships across users, tours, bookings, payments, invoices, suppliers, agents, and affiliates.

1. Core Domain Entity-Relationship Diagram

```
erDiagram
    ROLES ||--o{ USERS : "assigned to"
    ROLES ||--o{ ROLE_PERMISSIONS : "defines"
    PERMISSIONS ||--o{ ROLE_PERMISSIONS : "granted via"

    USERS ||--o| CUSTOMERS : "profiles"
    USERS ||--o| SUPPLIERS : "operates as"
    USERS ||--o| AGENTS : "operates as"
    USERS ||--o| AFFILIATES : "operates as"
    USERS ||--o{ USER_SESSIONS : "authenticates"
    USERS ||--o{ AUDIT_LOGS : "acts in"

    COUNTRIES ||--o{ STATES : "contains"
    STATES ||--o{ CITIES : "contains"

    COUNTRIES ||--o{ TOURS : "hosts"
    CITIES ||--o{ TOURS : "departs from"
    TOUR_CATEGORIES ||--o{ TOURS : "classifies"
    SUPPLIERS ||--o{ TOURS : "provides"

    TOURS ||--o| TOUR_OVERVIEWS : "details"
    TOURS ||--o{ TOUR_ITINERARIES : "schedules"
    TOURS ||--o{ TOUR_PRICING : "costs"
    TOURS ||--o{ TOUR_CALENDAR : "departs on"
    TOURS ||--o{ TOUR_DISCOUNTS : "applies"
    TOURS ||--o{ TOUR_VERSIONS : "versions"

    CUSTOMERS ||--o{ BOOKINGS : "places"
    AGENTS ||--o{ BOOKINGS : "reserves"
    SUPPLIERS ||--o{ BOOKINGS : "fulfills"
    TOURS ||--o{ BOOKINGS : "reserved for"
    AFFILIATE_ATTRIBUTIONS ||--o| BOOKINGS : "referred by"

    BOOKINGS ||--o{ BOOKING_TRAVELLERS : "includes"
    BOOKINGS ||--o{ BOOKING_OPTIONAL_ACTIVITIES : "addons"
    BOOKINGS ||--o{ BOOKING_ACCOMMODATIONS : "lodges"
    BOOKINGS ||--o{ BOOKING_STATUS_HISTORY : "tracks"
    BOOKINGS ||--o{ PAYMENTS : "paid via"
    BOOKINGS ||--o{ INVOICES : "billed with"
    BOOKINGS ||--o{ CANCELLATION_REQUESTS : "cancels"
    BOOKINGS ||--o{ TOUR_REVIEWS : "rates"

    PAYMENTS ||--o{ PAYMENT_TRANSACTIONS : "logs"
    PAYMENTS ||--o{ PAYMENT_HOLDS : "authorizes"

    SUPPLIERS ||--o{ SUPPLIER_LEDGERS : "credits"
    SUPPLIERS ||--o{ SUPPLIER_PAYOUTS : "requests"
    SUPPLIER_PAYOUTS ||--o{ SUPPLIER_PAYOUT_ITEMS : "aggregates"

    AGENTS ||--o{ AGENT_LEDGER : "earns"

    AFFILIATES ||--o{ AFFILIATE_LINKS : "shares"
    AFFILIATE_LINKS ||--o{ AFFILIATE_CLICKS : "tracks"
    AFFILIATE_CLICKS ||--o{ AFFILIATE_ATTRIBUTIONS : "attributes"
    AFFILIATES ||--o{ AFFILIATE_CONVERSIONS : "earns"
    AFFILIATES ||--o{ AFFILIATE_PAYOUTS : "withdraws"
```

2. Core Entity Descriptions & Primary Keys

Entity Name	Primary Key	Description & Role in Architecture
`users`	`id` (int)	Master user identity record containing email, password hash, status, 2FA flag, and lockout parameters.
`roles`	`id` (int)	System and custom RBAC roles (`super-admin` , `admin` , `supplier` , `agent-reseller` , `customer` , `affiliate`).
`permissions`	`id` (int)	Granular permission catalog mapping action (`get` , `post` , `put` , `delete`) to module slugs.
`customers`	`id` (int)	Customer profile linking to `users.id` , holding contact address, emergency contacts, and nationality.
`suppliers`	`id` (int)	Tour operator identity linking to `users.id` , tracking approval status, commission rates, and banking.
`agents`	`id` (int)	B2B travel agent profile linking to `users.id` , holding trade license, credit limits, and agency details.
`affiliates`	`id` (int)	Marketing affiliate profile linking to `users.id` , holding referral codes, tax IDs, and payout methods.
`tours`	`id` (int)	Core tour catalog record with slug, destination link, durations, starting price, and publishing status.
`tour_versions`	`id` (int)	Immutable snapshot of tour edits submitted for review; prevents live tour corruption before admin approval.
`tour_calendar`	`id` (int)	Individual departure calendar date entry tracking available seats, departure status, and cutoff rules.
`bookings`	`id` (int)	The central transaction hub of Tourvaa. Links tour, dates, customer, agent, supplier, and financial state.
`booking_travellers`	`id` (int)	Detailed passenger manifests containing full names, passport details, DOB, and dietary requirements.
`payments`	`id` (int)	Payment tracking record storing gateway transaction ID, captured amount, currency, and authorization status.
`payment_holds`	`id` (int)	Explicit record of pre-authorization holds held with Stripe/PayPal pending supplier booking acceptance.
`invoices`	`id` (int)	Formal commercial invoice issued to customers or agents, tracking invoice number, GST, and balance due.
`supplier_ledgers`	`id` (int)	Double-entry ledger tracking net tour amounts credited to suppliers, platform commission, and debits.
`agent_ledger`	`id` (int)	Commercial account statement tracking agent commissions, customer direct payments, and remittances.

`affiliate_links`	`id` (int)	Custom referral links (`/r/{code}`) configured with attribution window days and destination redirects.
`affiliate_conversions`	`id` (int)	Commission records generated upon successful checkout attribution, linked to booking financial records.

Authentication & Authorization Decision Flow

This document details the authentication and authorization lifecycle in Tourvaa, encompassing credential login, dual-domain JWT validation, 2FA with TOTP, session tracking, password reset token rotation, and RBAC permission checks.

1. Login & Token Lifecycle Sequence Diagram

```
sequenceDiagram
    autonumber
    actor User as User / Portal Visitor
    participant Browser as Web Browser
    participant FE as Next.js Frontend
    participant API as FastAPI Backend (/api/auth)
    participant DB as MySQL Database
    participant SMTP as SMTP Mailer

    User->>Browser: Enters Email & Password
    Browser->>FE: Submit Form Data
    FE->>API: POST /api/auth/login (email, password)

    API->>DB: Query User by Email
    alt User Not Found or Inactive
        API-->>FE: 401 Unauthorized ("Invalid credentials")
    end

    alt Account Locked (locked_until > now)
        API-->>FE: 403 Forbidden ("Account locked due to failed attempts")
    end

    API->>API: verify_password(plaintext, stored_bcrypt_hash)
    alt Password Invalid
        API->>DB: Increment failed_login_attempts
        opt failed_login_attempts ≥ 5
            API->>DB: Set locked_until = now() + 15 minutes
        end
        API-->>FE: 401 Unauthorized ("Invalid credentials")
    end

    API->>DB: Reset failed_login_attempts = 0

    alt 2FA Enabled on Account
        API-->>FE: 200 OK (requires_2fa: true, temp_token: JWT)
        FE->>User: Prompt for 6-Digit TOTP / Backup Code
        User->>FE: Enter OTP
        FE->>API: POST /api/auth/2fa/verify (temp_token, code)
        API->>API: Verify TOTP via pyotp or validate backup code
    end

    API->>API: Determine Role & Portal (admin vs normal domain)
    API->>API: Sign Access JWT (exp: 24h) with domain secret
    API->>API: Sign Refresh JWT (exp: 30d, jti: uuid4)

    API->>DB: Insert into user_sessions (user_id, jti, ip, user_agent, expires_at)
    API->>DB: Insert into login_history (status: success)

    API-->>FE: Set httpOnly cookies: 'tourvaa_access' & 'tourvaa_refresh'
    API-->>FE: Return user profile JSON

    FE->>FE: Update AuthProvider State
    FE->>Browser: Redirect to User Portal Dashboard
```

2. Authorization Decision Flowchart

```
flowchart TD
    START([Incoming API Request]) ⟶ EXTRACT[Extract Token from Cookie or Bearer Header]
    EXTRACT ⟶ TOKEN_EXISTS{Token Present?}

    TOKEN_EXISTS -- No ⟶ PUBLIC_CHECK{Is Route Public?}
    PUBLIC_CHECK -- Yes ⟶ ALLOW_PUBLIC[Execute Endpoint Handler]
    PUBLIC_CHECK -- No ⟶ DENY_401[401 Unauthorized]

    TOKEN_EXISTS -- Yes ⟶ DECODE[decode_auth_token: Check Admin & Normal Secrets]
    DECODE ⟶ SIGNATURE_VALID{Signature Valid?}
    SIGNATURE_VALID -- No ⟶ EXPIRED_CHECK{Signature Expired?}
    EXPIRED_CHECK -- Yes ⟶ DENY_401_EXP[401 Token Expired]
    EXPIRED_CHECK -- No ⟶ DENY_401_INV[401 Invalid Token]

    SIGNATURE_VALID -- Yes ⟶ FETCH_USER[Fetch User & Role from Database]
    FETCH_USER ⟶ USER_ACTIVE{user.is_active == 1?}
    USER_ACTIVE -- No ⟶ DENY_DEACTIVE[403 Account Deactivated]

    USER_ACTIVE -- Yes ⟶ VERSION_CHECK{token_version == user.token_version?}
    VERSION_CHECK -- No ⟶ DENY_REVOKED[401 Session Revoked - Password Changed]

    VERSION_CHECK -- Yes ⟶ SESSION_DB_CHECK{UserSession Active & Not Revoked?}
    SESSION_DB_CHECK -- No ⟶ DENY_SESSION_REVOKED[401 Session Terminated]

    SESSION_DB_CHECK -- Yes ⟶ APPROVAL_CHECK{Partner Role Requires Approval?}
    APPROVAL_CHECK -- Supplier ⟶ SUP_APPROVED{supplier.approval_status == 'approved'?}
    APPROVAL_CHECK -- Agent ⟶ AGT_APPROVED{agent.approval_status == 'approved'?}
    APPROVAL_CHECK -- Affiliate ⟶ AFF_APPROVED{affiliate.status == 'approved'?}
    APPROVAL_CHECK -- Other ⟶ PERMISSION_GATE

    SUP_APPROVED -- No ⟶ SAFE_SUP_CHECK{Prefix in PENDING_SUPPLIER_SAFE?}
    SAFE_SUP_CHECK -- Yes ⟶ PERMISSION_GATE
    SAFE_SUP_CHECK -- No ⟶ DENY_SUP_APP[403 SUPPLIER_APPROVAL_REQUIRED]

    AGT_APPROVED -- No ⟶ SAFE_AGT_CHECK{Prefix in PENDING_AGENT_SAFE?}
    SAFE_AGT_CHECK -- Yes ⟶ PERMISSION_GATE
    SAFE_AGT_CHECK -- No ⟶ DENY_AGT_APP[403 AGENT_APPROVAL_REQUIRED]

    AFF_APPROVED -- No ⟶ DENY_AFF_APP[403 AFFILIATE_APPROVAL_REQUIRED]

    SUP_APPROVED -- Yes ⟶ PERMISSION_GATE
    AGT_APPROVED -- Yes ⟶ PERMISSION_GATE
    AFF_APPROVED -- Yes ⟶ PERMISSION_GATE

    PERMISSION_GATE[Evaluate Endpoint Permission Requirement] ⟶ SUPER_ADMIN_CHECK{user.role == 'sup
    SUPER_ADMIN_CHECK -- Yes ⟶ GRANT_ACCESS[Allow Request Execution]

    SUPER_ADMIN_CHECK -- No ⟶ HAS_PERM{User Role has Required Permission Slug?}
    HAS_PERM -- Yes ⟶ GRANT_ACCESS
    HAS_PERM -- No ⟶ DENY_403[403 Forbidden: Insufficient Permissions]
```

3. Key Security Properties

1. **Dual Cryptographic Domains**:

- The administration interface uses `ADMIN_JWT_SECRET_KEY`.
- External portal users (customers, suppliers, agents, affiliates) use `NORMAL_JWT_SECRET_KEY`.
- Even if a customer obtains a valid token, it is cryptographically rejected by administrative endpoints because the token domain claim is strictly verified.

2. **Revocation via `token_version`**:

- When a user changes their password or clicks "Sign out everywhere", the database increments `user.token_version`.
- All outstanding JWTs become instantly invalid, preventing session replay.

3. **Defense Against Brute Force**:

- Consecutive login failures are counted in `failed_login_attempts`.
- Upon the 5th failed attempt, `locked_until` is populated with `now() + 15 minutes`.
- The API short-circuits attempts during lockout without verifying the password, mitigating timing attacks.

Booking Lifecycle & State Transition Workflow

This document models the complete booking lifecycle in Tourvaa, detailing the state machine, seat reservation mechanics, supplier acceptance requirements, payment hold-to-capture sequence, and customer receipt generation.

1. Booking State Transition Diagram

```
stateDiagram-v2
    [*] ⟶ draft: Customer initiates booking

    draft ⟶ pending_payment: Checkout started
    draft ⟶ cancelled: Abandoned

    pending_payment ⟶ payment_authorized: Gateway Pre-Auth Hold
    pending_payment ⟶ pending_supplier_assignment: Unassigned / Agent Booking
    pending_payment ⟶ cancelled: Stale Unpaid Expiry (60m)

    payment_authorized ⟶ pending_supplier_acceptance: Supplier Notified
    pending_supplier_assignment ⟶ pending_supplier_acceptance: Admin Assigns Supplier

    pending_supplier_acceptance ⟶ supplier_accepted: Supplier Accepts Request
    pending_supplier_acceptance ⟶ supplier_reassignment_required: Supplier Declines
    pending_supplier_acceptance ⟶ cancelled: 4-Day Hold Expiry Sweep

    supplier_reassignment_required ⟶ pending_supplier_assignment: Admin Reassigns
    supplier_reassignment_required ⟶ declined: No Alternative Supplier

    supplier_accepted ⟶ confirmed: Tourvaa Admin Confirms & Gateway Captures
    supplier_accepted ⟶ cancelled: Pre-Confirmation Cancellation

    confirmed ⟶ ready_to_travel: Voucher & Receipt Issued
    ready_to_travel ⟶ ongoing: Tour Departure Date Reached
    ongoing ⟶ completed: Tour End Date Reached

    confirmed ⟶ cancellation_requested: Customer / Agent Requests Cancellation
    cancellation_requested ⟶ confirmed: Cancellation Rejected
    cancellation_requested ⟶ cancelled: Cancellation Approved (Refund Applied)

    declined ⟶ refunded: Gateway Pre-Auth Released / Refunded
    cancelled ⟶ refunded: Funds Credited to Customer

    completed ⟶ [*]
    refunded ⟶ [*]
```

2. End-to-End Booking Sequence Walkthrough

```
sequenceDiagram
    autonumber
    actor Cust as Customer / Agent
    participant FE as Next.js Storefront / Portal
    participant API as FastAPI Backend
    participant DB as MySQL Database
    participant Stripe as Stripe Gateway
    participant Sup as Supplier Portal
    participant Admin as Tourvaa Operations
    participant Worker as Background Sweeper

    Cust->>FE: Select Tour & Travel Date
    FE->>API: GET /api/tours/{id}/availability-schedule
    API->>DB: Query TourCalendar (available_seats ≥ passenger_count)
    API-->>FE: Return Departure Slots & Pricing

    Cust->>FE: Fill Traveller Details & Click "Reserve"
    FE->>API: POST /api/checkout/create-session
    API->>DB: Insert Booking (status: 'draft', payment_status: 'unpaid')
    API->>DB: Decrement TourCalendar available_seats (Hold seats)
    API->>Stripe: Create Checkout Session (capture_method: 'manual')
    API-->>FE: Return Stripe Checkout URL & Booking Code

    Cust->>Stripe: Approves Card Authorization
    Stripe-->>API: Webhook (checkout.session.completed)
    API->>DB: Insert PaymentHold & Payment (status: 'authorized')
    API->>DB: Transition Booking → 'payment_authorized'
    API->>DB: Notify Assigned Supplier (Notification & Email)
    API->>DB: Transition Booking → 'pending_supplier_acceptance'

    alt Supplier Accepts within 4 Days
        Sup->>API: POST /api/bookings/{id}/supplier-decision (action: 'accept')
        API->>DB: Transition Booking → 'supplier_accepted'
        API->>Admin: Notify Tourvaa Operations ("Supplier accepted booking")

        Admin->>API: POST /api/bookings/{id}/confirm
        API->>Stripe: POST /v1/payment_intents/{id}/capture (Capture Authorized Hold)
        API->>DB: Update Payment (status: 'completed', amount_paid: full/deposit)
        API->>DB: Transition Booking → 'confirmed'
        API->>DB: Generate Confirmation Receipt PDF (confirmation_receipts.py)
        API->>DB: Generate Customer Invoice (invoices.py)
        API->>Cust: Dispatch Booking Confirmation Email with Attached PDF Receipt
        API->>DB: Credit Supplier Ledger (Net tour cost less commission)
    else Supplier Declines or Times Out
        Worker->>API: _payment_hold_loop() runs (every 60m)
        Worker->>Stripe: Cancel PaymentIntent (Release Card Authorization Hold)
        Worker->>DB: Release TourCalendar seats
        Worker->>DB: Transition Booking → 'cancelled' / 'declined'
        Worker->>Cust: Send Apology Email & Offer Alternative Dates
    end
```

3. Critical Business Rules Verified from Source Code

1. **Payment Authorization Is Not Final Confirmation**:

- As stated in `app/services/payments.py` line 21: *"A payment authorization is not a receipt and supplier acceptance is not the customer-facing final confirmation."*
- Stripe card authorizations use `capture_method="manual"` to place an authorized hold without debiting the customer immediately.

2. **Four-Day Supplier Decision Window**:

- The automated background worker `_payment_hold_loop()` periodically cancels authorizations before they expire on the gateway, preventing funds from hanging in customer bank accounts if a supplier is unresponsive.

3. **Seat Holding & Stale Booking Expiry**:

- Unpaid bookings in `pending_payment` hold seats in `TourCalendar`. The coroutine `_expire_stale_bookings_loop()` runs every 15 minutes and automatically releases seats if payment is not authorized within `BOOKING_EXPIRY_HOLD_MINUTES` (60 minutes).

4. **Immutable Confirmation Receipts**:

- Final booking receipts (`booking-confirmation-{id}-v2.pdf`) are stored in `private-docs/booking-confirmations/` upon initial confirmation. They are immutable snapshots and are never dynamically recomputed from mutable database rows on later download.

Payment Pre-Authorization, Hold & Capture Flow

This document details the payment gateway interaction architecture for Stripe and PayPal, demonstrating how funds are pre-authorized upon checkout, held in escrow during supplier evaluation, captured upon final confirmation, or released/refunded upon rejection or cancellation.

1. Payment Lifecycle Architecture Diagram

```
flowchart TD
    CHECKOUT[Customer Initiates Checkout] ⟶ SELECT_GATEWAY{Choose Payment Method}

    SELECT_GATEWAY -- Stripe Credit/Debit Card ⟶ STRIPE_SESSION[Stripe Checkout Session<br/>payment
    SELECT_GATEWAY -- PayPal Account / Card ⟶ PAYPAL_ORDER[PayPal v2 Order<br/>intent='AUTHORIZE']

    STRIPE_SESSION ⟶ CUST_AUTH_STRIPE[Customer Approves Card in Stripe]
    PAYPAL_ORDER ⟶ CUST_AUTH_PAYPAL[Customer Authorizes via PayPal]

    CUST_AUTH_STRIPE ⟶ STRIPE_WEBHOOK[Webhook: checkout.session.completed]
    CUST_AUTH_PAYPAL ⟶ PAYPAL_CAPTURE[GET /api/payments/paypal/capture]

    STRIPE_WEBHOOK ⟶ RECORD_HOLD[Insert PaymentHold & Payment status='authorized']
    PAYPAL_CAPTURE ⟶ RECORD_HOLD

    RECORD_HOLD ⟶ DECISION_GATE{Supplier Decision within 4 Days?}

    DECISION_GATE -- Accepted by Supplier & Confirmed by Admin ⟶ CAPTURE_FUNDS[Capture Authorized F
    DECISION_GATE -- Declined or Decision Window Expired ⟶ RELEASE_FUNDS[Release Authorized Hold]

    subgraph CAPTURE_FLOW["Payment Capture Workflow"]
        CAPTURE_FUNDS ⟶ DO_CAPTURE{Provider?}
        DO_CAPTURE -- Stripe ⟶ STRIPE_POST[POST /v1/payment_intents/{id}/capture]
        DO_CAPTURE -- PayPal ⟶ PAYPAL_POST[POST /v2/payments/authorizations/{id}/capture]
        STRIPE_POST ⟶ CAPTURE_SUCCESS{Success?}
        PAYPAL_POST ⟶ CAPTURE_SUCCESS
        CAPTURE_SUCCESS -- Yes ⟶ MARK_PAID[Update Payment status='completed'<br/>Generate Confirmat
        CAPTURE_SUCCESS -- No ⟶ CAPTURE_RETRY[Schedule in _payment_capture_retry_loop<br/>Retried e
    end

    subgraph RELEASE_FLOW["Authorization Release / Void Workflow"]
        RELEASE_FUNDS ⟶ DO_CANCEL{Provider?}
        DO_CANCEL -- Stripe ⟶ STRIPE_CANCEL[POST /v1/payment_intents/{id}/cancel]
        DO_CANCEL -- PayPal ⟶ PAYPAL_VOID[POST /v2/payments/authorizations/{id}/void]
        STRIPE_CANCEL ⟶ VOID_SUCCESS[PaymentHold status='released'<br/>Payment status='voided'<br/>
        PAYPAL_VOID ⟶ VOID_SUCCESS
    end

    subgraph REFUND_FLOW["Post-Capture Refund Workflow"]
        MARK_PAID -. Customer Cancels Later .-> REFUND_CHECK{Refund Rule Evaluation}
        REFUND_CHECK ⟶ EXEC_REFUND{Provider?}
        EXEC_REFUND -- Stripe ⟶ STRIPE_REFUND[POST /v1/refunds with idempotency key]
        EXEC_REFUND -- PayPal ⟶ PAYPAL_REFUND[POST /v2/payments/captures/{id}/refund]
        STRIPE_REFUND ⟶ REFUND_COMPLETE[Payment status='refunded'<br/>Debit Supplier Ledger<br/>Iss
        PAYPAL_REFUND ⟶ REFUND_COMPLETE
    end
```

2. Key Gateway Integration Mechanisms

2.1 Webhook Signature Validation

1. **Stripe Webhook (`construct_event`)**:

- Parses the `Stripe-Signature` header for timestamp `t` and signature `v1`.
- Computes HMAC-SHA256 signature using `STRIPE_WEBHOOK_SECRET` over `${t}.${raw_body}`.
- Rejects stale webhooks where `abs(time.time() - int(t)) > 300` (5-minute replay defense).
- Validates using constant-time string comparison (`hmac.compare_digest`).

2. **PayPal Webhook (`verify_webhook_signature`)**:

- Compares incoming transmission headers (`PAYPAL-AUTH-ALGO` , `PAYPAL-TRANSMISSION-ID` , `PAYPAL-CERT-URL` , `PAYPAL-TRANSMISSION-SIG` , `PAYPAL-TRANSMISSION-TIME`).
- Verifies against `PAYPAL_WEBHOOK_ID` configured for the production API endpoint.

2.2 Idempotency & Fault Tolerance

- All Stripe captures, voids, and refunds supply a unique `Idempotency-Key` (e.g. `capture-{payment.id}-{attempt}`).
- If gateway capture fails due to temporary network partitions after supplier acceptance, the record enters retry state and is automatically re-attempted by `_payment_capture_retry_loop()` every 15 minutes.

Production Deployment Architecture & Infrastructure Topology

This document details the production container topology, networking, storage persistence, reverse proxy integration, and CI/CD automation of the Tourvaa platform.

1. Production Deployment Topology Diagram

```
flowchart TD
    subgraph INTERNET["Public Internet"]
        USERS["Visitors, Customers & Partners"]
        ADMINS["Tourvaa Staff & Administrators"]
        WEBHOOKS["Stripe & PayPal Webhook Events"]
    end

    subgraph CLOUD_EDGE["Edge & Ingress (Cloudflare / Reverse Proxy)"]
        EDGE_ROUTER["Reverse Proxy / SSL Ingress (Nginx / Traefik / Cloudflare)<br/>TLS 1.3 Terminati
    end

    subgraph DOCKER_HOST["Production Container Host (Docker / Container Cluster)"]
        subgraph FRONTEND_CONTAINER["Frontend Service (tourvaa-admin-frontend)"]
            NODE_SERVER["Node.js 22 Standalone Server (server.js)<br/>Port 3000 | Non-Root user: next
        end

        subgraph BACKEND_CONTAINER["Backend Service (tourvaa-admin-backend)"]
            ENTRYPOINT["docker-entrypoint.sh<br/>Alembic upgrade head (retries 30x)"]
            UVICORN_PROC["Uvicorn ASGI Server (app.main:app)<br/>Port 8000 | Non-Root user: tourvaa<b
            ENTRYPOINT ⟶ UVICORN_PROC
        end

        subgraph PERSISTENCE_CONTAINERS["Persistence Services"]
            MYSQL_SRV[("MySQL 8.0+ Database Server<br/>Port 3306 | InnoDB Engine<br/>Volume: mysql_da
            REDIS_SRV[("Redis Server (redis:8.0)<br/>Port 6379 | Appendonly: yes<br/>Volume: redis_da
        end

        subgraph VOLUMES["Host Mounted Volumes"]
            STOR_VOL[("Host Volume: /data/storage<br/>Public asset uploads")]
            PRIV_VOL[("Host Volume: /data/private-docs<br/>Invoices, Receipts, Partner KYC")]
        end
    end

    subgraph EXTERNAL_SERVICES["External Cloud Infrastructure"]
        CLOUDINARY_CDN["Cloudinary Media CDN<br/>(Image optimization & public storage)"]
        SMTP_RELAY["SMTP Mail Relay<br/>(SendGrid / Google Workspace / Postmark)"]
        STRIPE_API["Stripe API & Webhook Services"]
        PAYPAL_API["PayPal Sandbox & Live REST API"]
    end

    %% Inbound connections
    USERS ⟶|HTTPS :443| EDGE_ROUTER
    ADMINS ⟶|HTTPS :443| EDGE_ROUTER
    WEBHOOKS ⟶|HTTPS :443| EDGE_ROUTER

    %% Edge to containers
    EDGE_ROUTER ⟶|HTTP :3000 (Storefront & Portals)| NODE_SERVER
    EDGE_ROUTER ⟶|WSS :8000 (/api/ws/messages)| UVICORN_PROC
    EDGE_ROUTER ⟶|HTTP :8000 (Webhook Endpoints)| UVICORN_PROC

    %% Container to container
    NODE_SERVER ⟶|Internal HTTP Proxy (/api/*)| UVICORN_PROC
    UVICORN_PROC ⟶|TCP :3306 (SQLAlchemy)| MYSQL_SRV
    UVICORN_PROC ⟶|TCP :6379 (Locks & WS)| REDIS_SRV
    UVICORN_PROC ⟶|Filesystem Mount| STOR_VOL
    UVICORN_PROC ⟶|Filesystem Mount| PRIV_VOL
```

```
    %% Backend to external
    UVICORN_PROC ⟶|REST API| CLOUDINARY_CDN
    UVICORN_PROC ⟶|SMTP TLS :465| SMTP_RELAY
    UVICORN_PROC ⟶|HTTPS API| STRIPE_API
    UVICORN_PROC ⟶|HTTPS API| PAYPAL_API
```

2. Infrastructure & Container Components

2.1 Edge Ingress & Reverse Proxy

- **TLS Termination**: All incoming traffic arrives on port 443 with modern TLS (TLS 1.2/1.3).
- **Reverse Proxy Routing**:
- `/*` routes forward to the Next.js frontend container on `http://frontend:3000`.
- `/api/ws/messages` WebSocket upgrade requests are routed directly to the backend on `http://backend:8000`.
- Direct webhook paths (`/api/payments/stripe/webhook`, `/api/payments/paypal/webhook`) can be routed directly to the backend to minimize hop latency.

2.2 Frontend Container (tourvaa-admin-frontend)

- **Base Image**: `node:22-alpine`.
- **Runtime User**: Runs as unprivileged user `nextjs:nodejs` (../../UID/GID 1001).
- **Standalone Build**: Leverages Next.js `output: "standalone"`, packing only necessary dependencies into `server.js` and reducing container size from ~1.2GB to <180MB.
- **Healthcheck**: Periodically polls `http://127.0.0.1:3000/` every 30 seconds.

2.3 Backend Container (tourvaa-admin-backend)

- **Base Image**: `python:3.12-slim`.
- **System Dependencies**: Includes `libgomp1` (required by ONNX runtime for FastEmbed embeddings).
- **Runtime User**: Runs as unprivileged user `tourvaa:tourvaa` (../../UID/GID 1001).
- **Entrypoint Script (`docker-entrypoint.sh`)**:
- Checks `RUN_MIGRATIONS=true`.
- Executes `python -m alembic upgrade head` with up to 30 retries, guaranteeing the database schema is up-to-date before Uvicorn starts.
- **Healthcheck**: Queries `http://127.0.0.1:8000/api/health` every 30 seconds.

2.4 Data Persistence Volumes

- **MySQL Volume (`/var/lib/mysql`)**: Ensures durable database records survive container updates.
- **Redis Volume (`/data`)**: Configured with `appendonly yes` for persistent distributed lock keys and rate-limit tracking.

- **Storage Volume (`/data/storage`)**: Houses static asset uploads served via `/storage`.
- **Private Docs Volume (`/data/private-docs`)**: Stores generated PDF invoices, itineraries, booking receipts, and partner business documents outside the public web root.

Affiliate Referral Tracking & Commission Workflow

This document details the tracking, attribution, commission calculation, and wallet payout mechanics of the Tourvaa Affiliate Marketing system.

1. Affiliate Referral & Attribution Architecture Diagram

```
sequenceDiagram
    autonumber
    actor Visitor as Prospective Traveller
    actor Affiliate as Marketing Partner
    participant ShortLink as Short Redirect (/r/{code})
    participant FE as Next.js Storefront
    participant API as FastAPI Backend
    participant DB as MySQL Database
    participant Worker as Commission Engine

    Affiliate->>API: POST /api/affiliate/links (create custom link)
    API->>DB: Insert AffiliateLink (code: 'PROMO2026', attribution_window: 30 days)
    API-->>Affiliate: Link Created: https://tourvaa.com/r/PROMO2026

    Visitor->>ShortLink: Clicks https://tourvaa.com/r/PROMO2026
    ShortLink->>DB: Insert AffiliateClick (ip, user_agent, referrer, clicked_at)
    ShortLink->>DB: Increment AffiliateLink.clicks_count
    ShortLink-->>Visitor: HTTP 307 Redirect to Destination URL with Cookie 'tv_aff_code'

    Visitor->>FE: Browses Tours & Decides to Book
    FE->>API: POST /api/checkout/create-session (Payload includes referral cookie)
    API->>DB: Query AffiliateLink by code
    API->>DB: Validate Attribution Window (click_time + window_days ≥ now)

    alt Attribution Valid
        API->>DB: Insert AffiliateAttribution (affiliate_id, link_id, booking_id)
        API->>DB: Set Booking.affiliate_attribution_id
    end

    Note over Visitor, Worker: Booking Confirmed & Payment Captured
    API->>Worker: Trigger Affiliate Commission Calculation
    Worker->>DB: Query AffiliateCommissionRule for Tour / Affiliate Tier
    Worker->>Worker: compute_commission(eligible_amount, rule_type, percentage/flat)
    Worker->>DB: Insert AffiliateConversion (status: 'pending_hold')

    Note over Worker, DB: Tour Completed & Dispute Window Elapsed (e.g. 14 Days)
    Worker->>DB: Transition Conversion status → 'approved'
    Worker->>DB: Credit Affiliate Wallet Balance (AffiliateWalletTransaction)
    Worker->>Affiliate: Email Notification: "Commission Earned"

    Affiliate->>API: POST /api/affiliate/payouts (request payout to bank/PayPal)
    API->>DB: Verify Wallet Balance ≥ payout_amount
    API->>DB: Deduct Wallet & Insert AffiliatePayout (status: 'pending')

    Admin->>API: POST /api/admin/affiliates/payouts/{id}/approve
    API->>DB: Update AffiliatePayout status → 'paid'
    API->>Affiliate: Dispatch Payout Confirmation Email
```

2. Commission Calculation Rules & Slabs

- **Rule Types**: Supports both `percentage` (e.g. 5% of net tour price) and `fixed` amounts (e.g. $50 per booking).
- **Attribution Logic**: First-touch or last-touch attribution within configurable attribution windows (default: 30 days).

- **Fraud Prevention**: Commissions remain in `pending_hold` status until the tour travel date has completed and cancellation dispute periods have expired. Cancelled or refunded bookings automatically invalidate associated pending conversions.

Realtime WebSocket Messaging & Notification Flow

This document details the WebSocket architecture in Tourvaa, explaining the ticket minting protocol, cross-worker Redis pub/sub messaging, presence tracking, and conversation thread synchronization.

1. WebSocket Connection & Messaging Architecture

```
sequenceDiagram
    autonumber
    actor Client as Browser Client
    participant Proxy as Next.js Edge Proxy
    participant REST as FastAPI REST API
    participant WS as FastAPI WebSocket Hub
    participant Redis as Redis Pub/Sub (ws:messages)
    participant Worker2 as Sibling Uvicorn Worker
    participant DB as MySQL Database

    Note over Client, REST: Step 1: Secure Handshake Authentication
    Client->>Proxy: POST /api/messaging/ws/ticket (with tourvaa_access cookie)
    Proxy->>REST: Forward to /api/messaging/ws/ticket
    REST->>REST: Authenticate user & mint ticket (30s TTL UUID)
    REST->>Redis: SETEX ws:ticket:{ticket} 30 {user_id, role, portal}
    REST-->>Client: Return JSON { ticket: "uuid4_single_use" }

    Note over Client, WS: Step 2: Direct WebSocket Connection
    Client->>WS: WebSocket Upgrade: wss://backend/api/ws/messages?ticket=uuid4
    WS->>Redis: GETDEL ws:ticket:{ticket} (Atomic Redemption)
    alt Ticket Invalid or Expired
        WS-->>Client: Close Socket (Code 4001: Unauthorized)
    end
    WS->>WS: Register connection in UserSocketRegistry[user_id]
    WS-->>Client: Accept Connection & Send Ready Event

    Note over Client, DB: Step 3: Message Posting & Cross-Worker Broadcast
    Client->>REST: POST /api/messaging/threads/{id}/messages (text, attachment)
    REST->>DB: Insert Message into database
    REST->>DB: Commit Transaction
    REST->>Redis: PUBLISH ws:messages { event: 'new_message', thread_id, payload }

    par Worker 1 (Hosting Sender Socket)
        Redis-->>WS: Ingest Pub/Sub frame
        WS->>Client: Send WebSocket JSON Frame (Message Delivered)
    and Worker 2 (Hosting Recipient Socket)
        Redis-->>Worker2: Ingest Pub/Sub frame
        Worker2->>Worker2: Lookup recipient user_id in local socket registry
        Worker2->>Client: Send WebSocket JSON Frame (Message Received Live)
    end
```

2. Technical Highlights

1. **Ticket-Based Handshake**:

- Web browsers cannot set custom headers during native `new WebSocket()` upgrades.
- Cross-origin WebSocket handshakes bypass the Next.js same-origin rewrite proxy.
- Tourvaa mints a single-use, 30-second Redis-backed ticket via an authenticated same-origin REST request, then passes it as a query param. The ticket is immediately destroyed upon connection (`GETDEL`).

2. **Multi-Worker Scaling via Redis**:

- In production environments where `WEB_CONCURRENCY > 1`, user sockets are distributed across different OS processes.
- The Redis pub/sub channel `ws:messages` guarantees that any message published by worker A immediately broadcasts to worker B holding the recipient's socket.

3. **Presence & Heartbeats**:

- Sockets emit periodic ping/pong frames (`PRESENCE_TTL_SECONDS = 75`).
- Connections are capped at `MAX_SOCKETS_PER_USER = 5` to prevent resource starvation.

Architectural Module Dependencies & Communication Graph

This document details the inter-module dependency graph, showcasing the communication pathways and data flow among all 20 business modules in the Tourvaa platform.

1. Complete System Module Dependency Graph

```
graph LR
    subgraph IDENTITY["Identity & Governance"]
        AUTH[01. Auth & Users]
        ROLES[Roles & Permissions]
        AUDIT[19. Audit Logging]
        SETTINGS[20. Settings & Currency]
    end

    subgraph CATALOG["Catalog & Inventory"]
        GEO[13. Destination Guides & Geo]
        TOURS[02. Tours & Catalog]
        VERSIONS[03. Tour Versions & Approval]
        VIATOR[17. External Day Tours]
    end

    subgraph COMMERCIALS["Commercial Hub & Partners"]
        SUPPLIERS[08. Suppliers & Ledger]
        AGENTS[09. Agents & Resellers]
        AFFILIATES[10. Affiliate Program]
        CUSTOMERS[11. Customer Management]
    end

    subgraph TRANSACTION["Transaction Engine"]
        BOOKINGS[04. Booking Engine]
        CHECKOUT[05. Checkout & Payments]
        INVOICES[06. Invoices & Receipts]
        REFUNDS[07. Cancellations & Refunds]
    end

    subgraph ENGAGEMENT["Engagement & Support"]
        CMS[12. Website CMS & Blog]
        MESSAGING[14. Realtime Messaging]
        NOTIF[15. Notification Engine]
        CHATBOT[16. AI Chatbot & RAG]
        REPORTS[18. Reports & Analytics]
    end

    %% Dependencies
    AUTH ⟶ ROLES
    AUTH ⟶ AUDIT

    TOURS ⟶ GEO
    TOURS ⟶ SUPPLIERS
    VERSIONS ⟶ TOURS
    VERSIONS ⟶ AUTH
    VIATOR ⟶ GEO

    BOOKINGS ⟶ TOURS
    BOOKINGS ⟶ CUSTOMERS
    BOOKINGS ⟶ SUPPLIERS
    BOOKINGS ⟶ AGENTS
    BOOKINGS ⟶ AFFILIATES

    CHECKOUT ⟶ BOOKINGS
    CHECKOUT ⟶ SETTINGS
    INVOICES ⟶ BOOKINGS
    INVOICES ⟶ CHECKOUT
```

```
    REFUNDS ──► BOOKINGS
    REFUNDS ──► CHECKOUT

    SUPPLIERS ──► BOOKINGS
    AGENTS ──► BOOKINGS
    AFFILIATES ──► BOOKINGS

    BOOKINGS ──► NOTIF
    BOOKINGS ──► MESSAGING
    MESSAGING ──► AUTH

    CHATBOT ──► TOURS
    CHATBOT ──► CMS
    CMS ──► GEO

    REPORTS ──► BOOKINGS
    REPORTS ──► CHECKOUT
    REPORTS ──► SUPPLIERS
    REPORTS ──► AGENTS
```

2. Structural Layering & Separation of Concerns

The architecture strictly adheres to a layered hierarchy:

1. **Foundation Tier**: Database engines, Redis cache, filesystem roots, encryption utilities (`app/utils/money.py` , `app/utils/distributed_lock.py`).

2. **Persistence Tier**: 138 SQLAlchemy declarative models in `app/models/` defining primary keys, indexes, foreign key constraints, and cascade policies.

3. **Domain Service Tier**: 56 business domain services in `app/services/` encapsulating business logic, calculation formulas, and external API requests.

4. **Interface / Transport Tier**: 51 FastAPI routers in `app/routers/` handling HTTP request deserialization, Pydantic validation, authorization dependencies, and JSON serialization.

5. **Client Presentation Tier**: Next.js 16 frontend consuming the API via same-origin proxied HTTP requests and direct WebSocket streams.